

Moving HIVE to the New Access Levels — Impact and Build Plan

Goal: replace today's five company roles and 83 permission switches with the model in `docs/downloads/hive-access-levels-and-permissions.xlsx`:

- **3 access levels:** Owner, Admin, Staff (HRC committee members are Staff with an “HRC Committee” preset)
- **18 categories**, each set to Off / View / Edit
- **Presets** that each agency names itself (“Supervisor”, “Manager”, “Lead”...)
- **Scope:** Whole agency, Assigned homes, Assigned staff, Assigned clients, or Only themselves
- **Job title** is display only and has no effect on access

Basis: code and migrations on this branch as of 2026-09-21. The live database may differ from `supabase/migrations/`, so Phase 0 is a set of SQL checks the human runs first.

1. The short answer

This is a **large but very doable** change. It touches almost every part of HIVE, because “who are you and what can you see” is checked everywhere. The good news:

1. **Most of the 83 switches don't do anything today.** Only about **24** permission keys are actually checked anywhere in the code. The other ~59 are switches on the profile screen that nothing reads. Deleting them costs nothing.
2. **The database mostly uses one helper for managers.** Most security rules (policies) call `is_org_admin_or_manager()`. Change what that one function checks, and hundreds of rules follow along without being touched one by one.
3. **Pieces of “scope” already exist.** Clients and profiles already have a `team_id` (home), staff already have caseloads (`staff_assignments`), there is already a “can this person see this client's medical info” check (`can_access_client_phi`), and there is a scope-assignments settings panel. Those are the building blocks for “Assigned homes / staff / clients”.

The one part that is genuinely hard, and needs care because it protects client medical data, is **enforcing scope inside the database** (Phase 4).

2. What it affects

2.1 Database

Thing	What it is today	Size
app_role enum	admin, manager, employee, super_admin, committee_member, program_manager	used on organization_members.role, invitations.role, role_permissions.role, role_change_audit_log, and as a parameter type in helper functions
is_org_admin_or_manager(org, user)	“is Owner, Program Manager or Supervisor”	~740 references across 135 migration files
has_org_role(org, user, role)	“has exactly this role”; mostly used as “is Owner”	~270 references across 33 files
is_org_member(org, user)	“belongs to this agency at all”	~380 references; unchanged by this plan
is_super_admin()	Old platform-admin check; now just calls is_hive_executive()	~350 references; not a company role, out of scope , but the super_admin enum value can finally be deleted
is_hrc_committee_member()	Reads hrc_committee_members table or role = 'committee_member'	becomes: table only, or preset = “HRC Committee”
can_access_client_phi(client)	Managers see all clients; staff see clients on their caseload	this becomes the heart of scope
can_view_staff_pii()	Who can see staff personal info	needs scope added
has_permission(user, org, perm)	Reads the 83-switch tables	only used by the Hosts / referrals rules in the database
role_permissions table	Per-agency copy of the 83-switch matrix per role	delete after migration
user_permission_overrides table	Per-person switch overrides	replace with per-person category overrides
permission_audit_log, role_change_audit_log	Audit trails	keep; record the new fields going forward
staff_groups, staff_group_members, profiles.scope_group_id, is_lead	Lead groups for compliance scope	fold into the new scope, or keep only for compliance targeting
restore_my_admin_role()	Old self-promotion function, already disabled	delete

Literal role words written directly inside SQL in the migrations: 'admin' ~240 times, 'super_admin' ~130, 'manager' ~70, 'employee' ~40, 'program_manager' 7, 'committee_member' 6. Each needs to be found in the **live** policies (Phase 0 lists them) and pointed at the new helpers.

2.2 App code

Area	What changes	Files (approx.)
src/lib/rbac.ts	The source of truth: role list, labels, ranks, home pages, the 83 keys, default matrix. Rewritten.	1 file, imported by ~35
Role checks written directly in code (role === "manager", .eq("role", ...), etc.)	Point at level or category checks	~160 files: src/lib (~100, a third of them in obligations/), src/routes (~31), src/components (~25), hooks, 2-3 edge functions
RequirePermission page guards and can() calls	Switch from 24 old keys to category checks, e.g. can("billing", "view")	~55 places
Server-side guards (requirePermission, requireRoleAtLeast, requireOrgMembershipAdmin, assertOrgAdmin)	Same, on the server	~10 files
Permission and role admin pages	/dashboard/roles, /dashboard/permissions, /dashboard/settings/team-access, /dashboard/settings/role-audit, /dashboard/hive-exec/permissions	Replace with one Access & presets page; keep the audit page
Employee profile	The 83-switch Permissions section, role dropdown, Leads group/Scope section	Replace with: Access level ▼, Preset ▼, 18 category overrides, scope pickers
Add employee / Upload roster / Invitations / Join	Role dropdowns (the “Employee / Manager / Admin” mismatch, bug F-6)	Level + preset pickers
Roster, badges, top bars	Show raw role words (EMPLOYEE, MANAGER)	Show the preset name (or job title) plus a small level badge
Where people land after login (ROLE_HOME)	Staff → phone app, HRC → HRC page	Owner/Admin → dashboard; Staff → phone app, unless their preset sets a different home page (the “HRC Committee” preset lands on the HRC page)
Compliance escalation (isAdminLevelRole) and admin scope (isAdminScopeRole)	Decide who gets escalations / scoped views	Level + scope
New-agency signup	Seeds role_permissions for a new agency	Seeds the default presets instead
Generated DB types (src/integrations/supabase/types.ts)	Mirrors the enum and tables	Regenerate
Tests and e2e mocks	Personas admin / manager / dsp; permission tests	Update to the new levels
Docs	CLAUDE.md, Employees docs, the Permission appendix	Update

Careful: `src/lib/med-attestation.ts` has its own `ROLE_LABEL` for medication administrators. It has the same name but nothing to do with company roles, so leave it alone.

3. The biggest trap: the word “admin”

Today the database value `admin` **means Owner**. In the new model, **Admin means managers**. If we just renamed things, every one of the ~240 places that says `'admin'` in SQL (plus many in code) would **silently hand full Owner power to every manager** unless someone found and fixed it.

So: don’t rename `app_role`. Build a new field next to it and then delete the old one.

- Add `organization_members.access_level` with the values `owner`, `admin`, `staff`.
- Move every check over to it.
- Finally **drop** the old `role` column and `app_role` type.

If anything was missed, it breaks loudly (an error) instead of quietly letting a manager do Owner things. Failing loudly is what you want around client medical data.

Old → new mapping (nobody gains or loses access on day one)

Today (role)	New access_level	Preset auto-created from today’s defaults
admin (shown as Owner)	owner	— (Owners always have everything)
program_manager	admin	“Program Manager”, scope = Whole agency
manager (shown as Supervisor)	admin	“Supervisor”, scope = Whole agency <i>(the agency can narrow it to homes later)</i>
employee (shown as Staff)	staff	“DSP”, scope = Only themselves + caseload
committee_member	staff	“HRC Committee” (home page = HRC page), scope = Assigned clients
super_admin	—	leftover; platform access is already <code>hive_executives</code>

Anyone who has per-person overrides today keeps them. They’re translated into category overrides using sheet 5 of the spreadsheet (old key → category), keeping the *highest* level any old key gave them in that category.

4. New database pieces

Minimal design: 1 new table. Everything else reuses what the live database already has (checked 2026-09-25).

Change	Type	Purpose
access_presets (org, name, access_level, scope_mode, home_page, categories_jsonb)	new table (the only one)	The agency's named presets. <code>categories</code> holds all 18 settings in one field, e.g. <code>{"billing":"view","scheduling":"edit"}</code> . <code>home_page</code> is optional (the HRC Committee preset lands on the HRC page).
organization_members.access_level	new column + small enum (owner, admin, staff)	The 3 levels
organization_members.custom_role_id → rename to preset_id, add FK to access_presets	reuse	The column already exists, unused (0 rows set, no FK).
organization_members.scope_mode	new column	Whole agency / Assigned / Only themselves. Filled from the preset, changeable per person.
organization_members.category_overrides_jsonb	new column	Per-person exceptions in the same shape as the preset's <code>categories</code> . Replaces <code>user_permission_overrides</code> (0 rows live).
scope_assignments (user, scope_type, scope_ref_id)	reuse , widen <code>scope_type</code> to add <code>home</code> and <code>staff</code> (keeps <code>client</code>)	Assigned homes, staff and clients, all in one existing table (0 rows live). One row per manager + person, so any client or staffer can be assigned to any number of managers, and each sees everything their own preset allows.
teams.manager_id	reuse	Copied into <code>scope_assignments</code> as <code>home</code> rows on day one.
Rule: at least one active Owner per agency	trigger on <code>organization_members</code>	Nobody can lock the agency out
Rule: Admins can't grant beyond themselves or edit Owners	in the server function that saves access	Stops privilege creep

Deleted in Phase 5: `role_permissions`, `user_permission_overrides`, the `app_role` type, `organization_members.role`. Net result: **+1 table, -2 tables**.

New helper functions (all `SECURITY DEFINER`, org-scoped like today's):

- `is_org_owner(org, user)`
- `is_org_admin(org, user)`, which is true for Owner **or** Admin (the new meaning of “leadership”)

- `has_category(user, org, category, 'view' | 'edit')`, which checks override → preset → level ceiling
- `can_see_client(user, client_id)`, which applies whole agency / assigned homes (client's `team_id`) / assigned clients / own caseload
- `can_see_staff(user, staff_user_id)`, which applies whole agency / assigned homes (staff's `team_id` or scheduled there) / assigned staff / self

Code layout (small files, one job each):

<code>supabase/migrations/</code>	one migration per step, named for what it does
<code>src/lib/access/</code>	
<code>levels.ts</code>	AccessLevel, ScopeMode types + level ceilings
<code>categories.ts</code>	the 18 categories + their Off/View/Edit explanations
<code>can.ts</code>	pure check: override → preset → level ceiling (unit tested)
<code>scope.ts</code>	pure helpers for whole agency / assigned / self
<code>access.functions.ts</code>	server functions: read my access, save a person's access, save a preset
<code>src/components/access/</code>	
<code>category-row.tsx</code>	one category with the explanation drop-down
<code>access-section.tsx</code>	Level ▾, Preset ▾, overrides, scope pickers (profile)
<code>presets-page.tsx</code>	Settings → Access & presets

The one-line trick for Phase 2: re-point the body of `is_org_admin_or_manager()` at `access_level IN ('owner', 'admin')`, and `has_org_role(..., 'admin')` at `access_level = 'owner'`. With the mapping above, that is **behavior-neutral**: the same people pass the same checks. Hundreds of policies are switched over without editing any of them.

5. The build, phase by phase

Each phase ships on its own and leaves the app working. Commits stay small (Lovable co-edits this repo). All SQL goes to the human through `docs/SQL_HANDOFF.md`.

Phase 0 — Look before touching (SQL handoff only, no changes)

The paste-ready version (7 queries, one row each, with what to expect) is at the top of `docs/SQL_HANDOFF.md` under “CHECK — Access levels Phase 0”. The sketch below shows the intent:

```
-- A. How many people hold each role today, per agency
select string_agg(organization_id || ':' || role || '=' || n, ' | ' order by organization_id, role)
from (select organization_id, role::text, count(*) n from organization_members where active group by 1,2) t;

-- B. Live enum values
```

```

select string_agg(enumlabel, ', ' order by enumsortorder) from pg_enum
where enumtypid = 'public.app_role'::regtype;

-- C. Every live policy that mentions a role word or a role helper (table :: policy)
select count(*) as policies, string_agg(tablename || ' :: ' || policyname, ' | ' order by tablename)
from pg_policies where schemaname = 'public'
    and (coalesce(qual, '') || coalesce(with_check, '')) ~*
        '(is_org_admin_or_manager|has_org_role|'admin'|'manager'|'employee'|'program_manager'|'committee_member')';

-- D. Live bodies of the role helpers
select string_agg(p.proname || ': ' || pg_get_functiondef(p.oid), E'\n-----\n')
from pg_proc p join pg_namespace n on n.oid = p.pronamespace
where n.nspname = 'public'
    and p.proname in
        ('is_org_admin_or_manager', 'has_org_role', 'is_hrc_committee_member', 'can_access_client_phi', 'can_view_staff_pii', 'has_permission')
;

-- E. Which per-person overrides exist (to translate)
select count(*), string_agg(distinct permission, ', ') from user_permission_overrides;

-- F. Do clients and staff reliably have a home (team_id)? Scope depends on it.
select (select count(*) from clients where team_id is null) as clients_without_home,
       (select count(*) from profiles where team_id is null) as profiles_without_home;

```

Phase 0 results (run 2026-09-25 on the live database):

Check	Result	What it means for the build
People per role	Only <code>admin</code> (Owner) and <code>employee</code> (Staff) are in use. TNS has 2 Owners and 4 Staff; the other 18 agencies are test workspaces with 1 Owner each (two are both named “Thrive Living”). Nobody is a Supervisor, Program Manager or Committee Member.	The move is simple: every current person becomes Owner or Staff. No manager exists yet, so nobody can be wrongly mapped. The Program Manager / Supervisor presets are still created, as templates.
Role values	Match the migrations (6 values).	No surprises.
Helper functions	<code>is_org_admin_or_manager</code> includes <code>program_manager</code> , <code>manager</code> and <code>admin</code> . <code>can_view_staff_pii</code> allows only the person themself, Owners, HIVE staff, and the home’s manager (<code>teams.manager_id</code>).	F-10 is confirmed withdrawn on the live database. Today, managers other than a home’s named manager can’t see staff SSN, date of birth, address or pay rates; the new model keeps that locked behind Payroll / Staff roster plus scope. <code>teams.manager_id</code> gets copied into Assigned homes.

Check	Result	What it means for the build
Policies using helpers	763 policies total: 312 use <code>is_org_admin_or_manager</code> , 59 <code>has_org_role</code> , 7 <code>has_permission</code> , 4 the HRC helper.	Re-pointing those four helpers in Phase 2 switches ~380 policies at once.
Policies with role words written in	65 policies across 47 tables.	Hand-edit list for Phase 2. About 10 of those tables (<code>organization_members</code> , <code>invitations</code> , <code>role_permissions</code> , <code>user_permission_overrides</code> , audit logs, <code>scope_assignments</code>) are the role system itself and get replaced anyway.
Per-person overrides	0.	Skip the override-translation step entirely.
Homes set	0 of 4 clients and 0 of 24 active members have a home (<code>team_id</code>).	“Assigned homes” won’t show anyone until homes are assigned on the Homes & Teams board (that screen already exists). Assigned staff and Assigned clients work without it. Assign homes before turning on home-based scope for a real manager.

Decided (2026-09-25):

- No Guest level. HRC committee members are **Staff** with an “HRC Committee” preset.
- Clients and staff can each be assigned to **multiple managers**. Every assigned manager sees the full record, within what their preset allows. Access is not limited to the service codes delivered in that manager’s home.

Still open: the final category/preset settings, which will come from the edited spreadsheet.

Phase 1 — Add the new pieces next to the old ones

- Migration: create the `access_level` enum and the `access_presets` table, add the new `organization_members` columns (rename `custom_role_id` → `preset_id`), widen `scope_assignments.scope_type`, and add the helper functions and the at-least-one-Owner trigger (§4).
- Backfill `access_level` and `preset_id` from `role` using the mapping in §3. Create the default presets per agency from today’s `role_permissions`, so each agency’s customizations carry over.
- ~~Translate `user_permission_overrides`~~: not needed, the live table is empty (Phase 0).
- Copy each home’s `teams.manager_id` into `scope_assignments` as `home` rows.
- Server functions that change roles (hire, invite accept, role change, roster upload) write **both** `role` and `access_level` for now.
- **Nothing visible changes.**

Phase 2 — Point the database at the new field (behavior-neutral)

- Re-point `is_org_admin_or_manager`, `has_org_role`, `is_hrc_committee_member` and `has_permission` (Hosts) at `access_level` / categories, as described in §4.
- Using the list from Phase 0 query C, replace every live policy that writes a role word directly (`role = 'admin', role IN (...)`) with `is_org_owner` / `is_org_admin` / `has_category`.
- **Check:** re-run Phase 0 query C; the only matches left should be the helpers themselves.

Phase 3 — Switch the app over

- Rewrite `rbac.ts`: `AccessLevel`, the 18 `Category` names, a `PresetLevel` type, `can(category, "view" | "edit")`, `LEVEL_HOME`. Delete the old role list, ranks, 83 keys and default matrix.
- Replace the ~24 old permission keys and the ~160 files with direct role checks with level or category checks. Use sheet 5 of the spreadsheet as the translation table.
- UI:
 - a new **Settings** → **Access & presets** page (create, rename, copy presets; edit 18 categories; default scope), replacing Roles, Permissions and Team access
 - a new **Access** section on the employee profile: Level ▼, Preset ▼, overrides, and scope pickers for homes, staff and clients
 - hire wizard, roster upload, invitations and join use Level + Preset
 - roster and badges show the preset name
 - **every permission explains itself** (see “Permission explanations” below)
- Signup seeds default presets.

Permission explanations (required). Wherever a category can be set (the Access & presets page and the employee profile’s Access section), each row has:

- A chevron that expands the row to show **what it covers** plus what the person **can** and **can’t** do at Off, View and Edit. The currently selected setting is highlighted. Wording comes from sheet 7 of the spreadsheet (“What each setting means”).
- The Off / View / Edit picker itself shows the same one-line explanation under each option, so nobody has to guess before choosing.
- For Staff-level presets, a line saying the access applies inside the phone app. For Agency settings, the text “Owners only — can’t be given to anyone else” instead of a picker.
- A footer on every expanded row: “Applies only to people in this person’s scope: .”

```
Billing                                [ View ▼ ]  ~
Covers: billing, claims, authorizations (1056), service codes.
○ Off  Can't see billing, claims, authorizations or service codes.
● View Can see billing, claims, authorizations and service codes.
      Can't change or submit anything.
```

○ Edit Can manage billing and claims, authorizations and service codes.
Applies only to people in this person's scope: Maple House.

The text lives in one catalog in code (`src/lib/access-categories.ts`: key, label, covers, off/view/edit text). The UI, the preset seeds and the tests all read from it, so the explanation can never drift from what the check actually does. A unit test fails if a category is missing any of its explanations. - **Check:** unit tests for `can()` and the old → new translation; e2e with one persona per level.

Phase 4 — Enforce scope in the database (the careful one)

- Extend `can_access_client_phi` to call `can_see_client`, and `can_view_staff_pii` to call `can_see_staff`.
- Add scope to the read rules of the tables with personal or client data. Go one domain per migration, and test each before the next:
 1. clients and client chart tables (goals, documents, medical, meds / eMAR)
 2. shifts, EVV timesheets
 3. shift notes, daily logs, form submissions, HHS daily records
 4. incidents, HRC
 5. billing (authorizations / `client_billing_codes`, claims), payroll
 6. staff profiles, staff file / obligations, employee loans
- Server functions that use the service role (they skip database rules) must call the same `can_see_client` / `can_see_staff` checks themselves.
- **Check:** a “Maple House manager” test account can see Maple House clients, staff, billing and timesheets, and gets nothing (not an error, just no rows) for Oak House. Run this for every domain.

Phase 5 — Delete the old terminology

- **Database:** drop `organization_members.role`, `invitations.role`, `role_permissions`, `user_permission_overrides` (after a grace period), `restore_my_admin_role()`, `has_org_role()`, and the `app_role` type. Postgres can't remove individual enum values, so the whole type goes once nothing uses it. Optionally rename `is_org_admin_or_manager` → `is_org_admin` at the same time.
 - **Code:** delete `ROLE_RANK`, `PROVIDER_ROLES`, `DEFAULT_MATRIX`, `PERMISSION_SECTION_MAP`, `profile-permission-groups.ts`, `staff-permission-toggles.ts`, `permissions-can.ts`, `isAdminLevelRole`, `isAdminScopeRole`, the old Roles/Permissions/Team-access pages, and the hard-coded “Employee / Manager / Admin / Supervisor / Program Manager / Committee Member” labels. Those names now live only as **preset names the agency typed in**.
 - Remove the 4 dead switches (`manage_roles`, `manage_organization` legacy; `view_platform_metrics`, `manage_all_orgs` are HIVE-staff only via `hive_executives`).
 - Regenerate `types.ts`. Update `CLAUDE.md` (RLS helper names, role vocabulary), the Employees docs, and e2e mocks.
 - “Employee” → “Staff” in the interface. The phone-app URL `/employee` can stay; users never see it.
-

6. Risks and how each is handled

Risk	Guard
"admin" changes meaning, and a missed check makes managers Owners	New field + drop the old one (\$3), so misses fail loudly
The live database doesn't match the migrations	Phase 0 reads the live policies and functions before any change
An agency loses all Owners	Database trigger requires at least one active Owner
A manager widens their own access	Only Owners change levels, presets and scope; Admins can't grant beyond themselves
Scope leaks client medical data between homes	Phase 4 goes one domain at a time with the Maple/Oak test, and service-role server functions call the same checks
Lovable edits the same files mid-change	Small atomic commits per phase; one writer at a time
Someone with custom overrides loses access	Overrides are translated, not dropped; audit log records before/after

7. Correction to the Employees docs

The Employees tab docs listed **F-10: Program Managers can't use Employee Loans because `is_org_admin_or_manager` excludes them.** That's wrong for the current code. Migration `20260825020000_replace_is_super_admin_with_hive_executive.sql` redefines the helper as `role IN ('admin', 'program_manager', 'manager')`, so Program Managers **are** allowed. It could only still happen if the live database never received that migration; Phase 0 check 3 (run 2026-09-25) confirmed the live helper includes `program_manager`. The Employees docs have been updated.